Data Integration e Pipelines

Aula 4 — Apache Airflow, Kubernetes e Orquestração

Prof. Rafael S Novo Pereira

FIAP — MBA Data Engineering

A jornada até aqui: 4 aulas, 4 ferramentas

Aula 1

Databricks

Pipeline
Bronze/Silver/Gold
com PySpark

Aula 2

Snowflake

SQL, Tasks, Streams
Azure Blob externo

Aula 3

dbt Cloud

Modelos versionados
Testes de qualidade

Aula 4

Airflow

Orquestração de
ponta a ponta

Cada ferramenta é excelente no que faz. Mas nenhuma, sozinha, resolve o fluxo completo.

O que cada aula resolveu — e o gap que sobrou

✓ Databricks: Transformar dados com código	✕ Tudo manual, sem schedule
✓ Snowflake: Automação interna (Tasks)	✕ Só orquestra dentro do Snowflake
✓ dbt Cloud: Qualidade como código	✕ Só transforma, não ingere nem notifica

Quem orquestra o fluxo inteiro? Quem conecta GitHub + Snowflake + dbt + email + retry?

Sem orquestrador vs com orquestrador

Sem orquestrador

- ✕ Aluno aperta play manualmente
- ✕ Se falha às 3h, ninguém sabe
- ✕ Sem retry automático
- ✕ Sem log centralizado
- ✕ Cada ferramenta isolada
- ✕ "Funciona no meu notebook"

Com Apache Airflow

- ✓ Pipeline roda no schedule definido
- ✓ Alerta por email/Slack se falhar
- ✓ Retry automático configurável
- ✓ Logs de cada task no browser
- ✓ DAG define ordem e dependências
- ✓ Orquestra Snowflake, dbt, APIs

Quem usa Airflow no Brasil?

E no mundo: Spotify, Uber, Netflix, Airbnb, Twitter...

Nubank

1000+ DAGs processando dados de 90M+ clientes

Itaú

Pipelines de risco e compliance regulatório

Mercado Livre

Orquestra dados de milhões de transações/dia

Airflow é o padrão de mercado. Saber Airflow hoje é como saber SQL há 10 anos.

O que é uma DAG?

Analogia: uma receita de bolo

Receita de bolo

1. Separar ingredientes
2. Misturar secos (depende de 1)
3. Misturar líquidos (depende de 1)
4. Juntar tudo (depende de 2 e 3)
5. Assar (depende de 4)
6. Verificar com palito (depende de 5)

DAG do Airflow

1. download_csvs
2. load_bronze (depende de 1)
3. transform_silver (depende de 2)
4. build_gold (depende de 3)
5. quality_checks (depende de 4)
6. Se falhar → retry automático

DAG = receita com ordem. Não pode assar antes de misturar. Não pode transformar antes de carregar.

Apache Airflow em números

12M+	10K+	2.4K+	v3.1
downloads/mês	empresas	contribuidores	março 2026

Criado pelo Airbnb em 2014. Open source. Projeto top-level Apache Foundation.

Airflow não processa dados. Ele orquestra quem processa.

É o maestro da orquestra. Databricks, Snowflake e dbt são os músicos.
Cada um toca seu instrumento muito bem, mas sem o maestro, ninguém sabe quando entrar.

Arquitetura do Airflow

Analogia: uma cozinha profissional

Webserver

O menu

Interface gráfica (UI)

Scheduler

O chef

Decide o que rodar e quando

Executor

Os cozinheiros

Executa as tasks (pods K8s)

Metadata DB

O caderno

Histórico, estado, configs

Por que estamos usando o Astronomer

A plataforma managed da Astronomer para Apache Airflow

Airflow na nuvem

Não precisa instalar nada. Abre o browser e usa.

Kubernetes por baixo

Cada task roda num container isolado. Escalável.

Trial gratuito

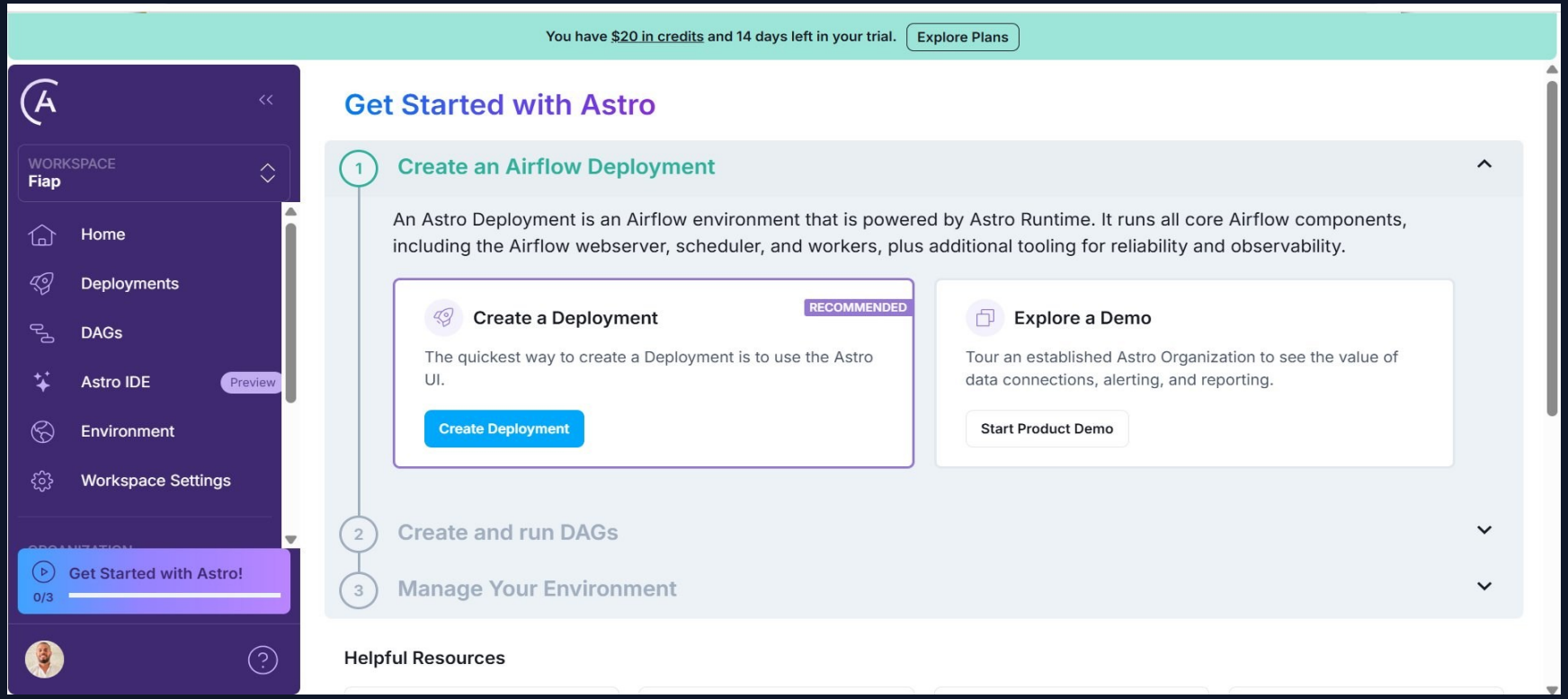
14 dias, US$20 créditos. Sem cartão de crédito.

Git integrado

Conecta o GitHub. Faz push, o Astro faz deploy.

Astro é para o Airflow o que o Snowflake é para SQL: a versão managed, sem dor de cabeça com infra.

Astro: o que vocês vão ver



Astronomer Astro — $20 créditos, 14 dias trial, Airflow 3.1

Pipeline Olist: o fluxo completo

GitHub

7 CSVs do Olist
~50 MB
Dados reais 2016-2018
100K pedidos

Airflow (Astro)

`download_and_load`

`transform_silver`

`build_gold`

`quality_checks`

Snowflake

Bronze (7 tabelas)
Silver (JOINs)
Gold (métricas)
Quality checks

O código: uma DAG em Python

```python
with DAG(
    dag_id="pipeline_olist",
    schedule=None,
    catchup=False,
) as dag:

    t1 = PythonOperator(download)
    t2 = PythonOperator(silver)
    t3 = PythonOperator(gold)
    t4 = PythonOperator(quality)

    t1 >> t2 >> t3 >> t4
```

Python puro. Sem XML, sem YAML, sem interface complicada.

>> define a ordem.
Se t1 falhar, t2 não roda.

schedule=None → manual
'0 6 * * *' → 6h todo dia

Schedule: quando o pipeline roda

O agendamento está no código — versionado no Git

`schedule=None`	Manual — aperta play
`"0 6 * * *"`	Todo dia às 6h
`"*/1 * * * *"`	A cada 1 minuto (demo)
`"0 */2 * * *"`	A cada 2 horas
`"0 22 * * TUE,THU"`	Terça e quinta às 22h

EXERCÍCIO: Escrevam a expressão cron para rodar às terças e quintas às 22h.

O que acontece quando o pipeline falha?

Retry automático

retries=2, retry_delay=60s
API caiu? Tenta de novo.

SLA e timeout

execution_timeout=30min
Demorou demais? Mata a task.

Alertas

email_on_failure=True
Time recebe email se falhar.

Backfill

Reprocessar dados do passado.
A máquina do tempo dos dados.

Backfill: a máquina do tempo dos dados

Cenário real: Nubank descobre em 15/mar que o cálculo de score de crédito estava errado desde 01/mar.

01/mar	05/mar	10/mar	15/mar
reprocessar	reprocessar	reprocessar	bug descoberto

O Airflow re-executa a DAG para cada dia de 01 a 14/mar. Sem reescrever código.
São 14 execuções automáticas. Cada uma baixa, transforma e valida os dados daquele dia.

EXERCÍCIO: O Itaú descobre que o pipeline de compliance está errado desde janeiro. Quantas execuções de backfill precisam? O que acontece com os dashboards durante o reprocessamento?

Kubernetes: por que o Airflow roda em containers

Isolamento

Cada task = 1 container. Dependências não conflitam.

Escalabilidade

Precisa de mais poder? K8s cria mais pods.

Eficiência

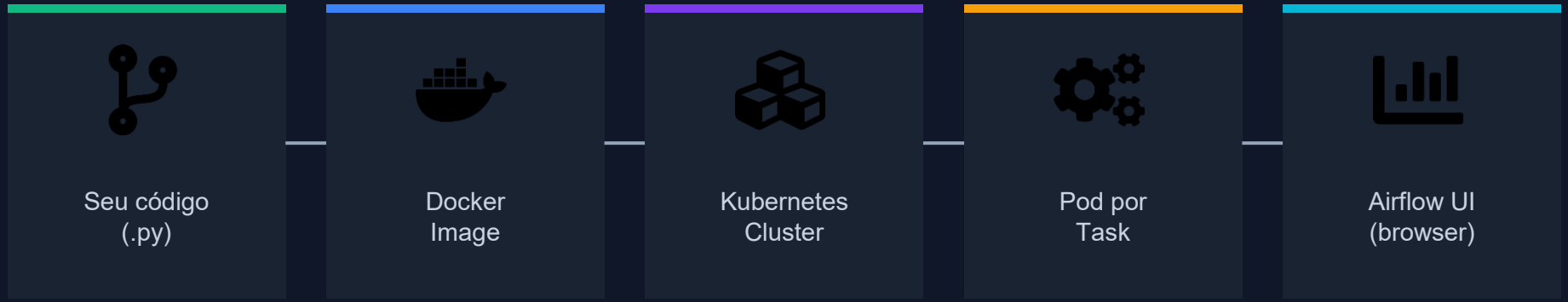
Pod nasce, roda a task e morre. Paga só o que usa.

Resiliência

Pod crashou? K8s detecta e o Airflow faz retry.

No Astro, isso é automático. Vocês não configuram K8s. Mas é bom saber o que está acontecendo por baixo.

Astro: vocês já estão usando Kubernetes



A prova real: quando separei download e load em tasks diferentes, o arquivo /tmp não existia no outro pod. Isso é K8s funcionando — cada task num container isolado com seu próprio filesystem.

Tudo isso acontece com um clique em "Deploy Project". O Astro abstrai toda a infraestrutura.

O que torna um pipeline robusto?

O checklist que separa amador de profissional

Idempotência

Rodar 2x produz o mesmo resultado. DROP + CREATE garante.

Retry

Se a API caiu, tenta de novo. Não precisa de intervenção humana.

Alertas

Se falhou, alguém precisa saber. Email, Slack, PagerDuty.

Backfill

Corrigir o passado sem reescrever código.

Quality checks

Validar ANTES de servir. Não confiar cegamente nos dados.

Observabilidade

Logs, métricas, duração. Saber o que está acontecendo.

Nosso pipeline Olist implementa TODOS esses 6 pilares.

Pipeline real: todas as tasks verdes

Dag: pipeline_olist_completo
Execução do Dag: manual__2026-03-26T21:04:12.354661+00:00

Pesquisar Dags Ctrl+K | Acionar

Opções

00:02:53
00:01:26

- inicio
- download_and_load_bronze
- transform_silver
- build_gold
- quality_checks
- fim
- download_csvs_github
- load_bronze_snowflake

manual__2026-03-26T21:04:12.354661+00:00 — Sucesso

Adicionar uma nota | Limpar Execução do Dag | Marcar Execução do Dag como...

Data Lógica	Tipo de Execução	Data Inicial	Data Final	Duração	Nome do Usuário que Disparou	Versão(s) do Dag
2026-03-26 18:04:10	manual	2026-03-26 18:04:13	2026-03-26 18:06:02	00:01:49	cmn7ncimw00bp01muremo7pdk	v2

Instâncias de Tarefa | Asset Events | Log de Auditoria | Código | Detalhes

Pesquisar Tarefas | Todos os Estados

6 Instâncias de Tarefa

ID da Tarefa	Índice do Mapa	Estado	Data Inicial	Número de Tentativas	Operador	Duração
fim		Sucesso	2026-03-26 18:06:01	1	EmptyOperator	
quality_checks		Sucesso	2026-03-26 18:05:55	1	PythonOperator	00:00:05.551
build_gold		Sucesso	2026-03-26 18:05:47	1	PythonOperator	00:00:06.030
transform_silver		Sucesso	2026-03-26 18:05:38	1	PythonOperator	00:00:07.249
download_and_load_bronze		Sucesso	2026-03-26 18:04:36	1	PythonOperator	00:00:59.830
inicio		Sucesso	2026-03-26 18:04:13	1	EmptyOperator	

Pipeline Olist executado hoje — Airflow 3.1 em Kubernetes no Astro

Resultado: dados reais no Snowflake

```
1  SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO LIMIT 5;
2
```

Add to Chat | Explain | Quick edit | Format

Results (just now)

Table | Chart

5 rows 86ms

	ESTADO	TOTAL_PEDIDOS	RECEITA_TOTAL	TICKET_MEDIO	DIAS_ENTREGA_MEDIO	PCT_ATRASO	_ATUALIZADO_EM
1	SP	41746	5998226.96	143.69	8.7	5.72	2026-03-26 14:05:50.714 -0700
2	RJ	12852	2144379.69	166.85	15.2	12.95	2026-03-26 14:05:50.714 -0700
3	MG	11635	1872257.26	160.92	11.9	5.48	2026-03-26 14:05:50.714 -0700
4	RS	5466	890898.54	162.99	15.2	6.99	2026-03-26 14:05:50.714 -0700
5	PR	5045	811156.38	160.78	11.9	4.88	2026-03-26 14:05:50.714 -0700

SP lidera com R$ 6M em receita. RJ tem 12.95% de atraso. Ninguém tocou no Snowflake — o Airflow fez tudo.

Panorama: ferramentas de orquestração

Airflow	Python DAGs, extensível, comunidade enorme	Curva de aprendizado
Prefect	Python nativo, deploy fácil	Comunidade menor
Dagster	Assets, tipagem forte	Mais recente
dbt Cloud	Schedule nativo para SQL	Só transforma
SF Tasks	Nativo, sem ferramenta externa	Só dentro do Snowflake

Quem sabe Airflow, aprende Prefect/Dagster em dias. O conceito é o mesmo.

Exercício: desenhem a DAG

Cenário: GOL Linhas Aéreas — pipeline diário de voos

O pipeline precisa:

1. Baixar dados de voos da API da ANAC
2. Baixar dados de clima do INMET (pode rodar em paralelo com 1)
3. Cruzar voos com clima — depende de 1 e 2
4. Calcular taxa de atraso por aeroporto
5. Gerar alerta se taxa > 30%
6. Enviar relatório por email

Desenhem o DAG. Quais tasks rodam em paralelo? (5 min)

Pitch de consultoria

Formato

3 minutos de pitch

2 minutos de perguntas

One-page com:

- Cenário de negócio
- Diagrama de arquitetura
- Ferramentas escolhidas
- Qualidade e operação

Critérios

Cenário de negócio	15%
Diagrama de arquitetura	25%
Ferramentas e justificativas	25%
Qualidade e operação	20%
Comunicação e clareza	15%

Parem e pensem: de onde vocês vieram

4 semanas atrás, vocês não sabiam o que era um DAG, um Operator, um backfill.
Hoje vocês:

- ✓ Construíram pipelines Bronze/Silver/Gold em 2 plataformas diferentes
- ✓ Orquestraram um pipeline real com dados de 100K pedidos brasileiros
- ✓ Viram retry, backfill e quality checks funcionando na prática
- ✓ Usaram Kubernetes em produção (mesmo sem saber kubectl)
- ✓ Podem explicar num pitch por que o Airflow é a cola que conecta tudo

Isso é mais do que muitos engenheiros de dados seniores sabem articular.
Vocês estão prontos.

Material e próximos passos

Repositório

github.com/rafsp/Aula2603_MBA

Contém:
- CSVs do Olist (7 arquivos)
- Código da DAG completa
- Dockerfile + requirements
- Guia passo a passo (Word)

Lab pós-aula

Sigam o Guia Visual (.docx)
passo a passo com screenshots:

1. Criar conta no Astro
2. Criar Deployment
3. Conectar GitHub
4. Deploy do código
5. Configurar Snowflake
6. Executar o pipeline

Data Integration e Pipelines

16 horas — o que vocês construíram:

2 plataformas de dados	Databricks (PySpark) + Snowflake (SQL)
Transformação com dbt	Modelos SQL, testes, documentação
Orquestração com Airflow	DAGs, K8s, operação em produção
Dados reais brasileiros	Olist (100K pedidos) + TPC-H (60M rows)
Kubernetes por baixo	Containers isolados, escaláveis, eficientes